Apuntes – Persistencia local con Room

Desarrollo de Aplicaciones I · UADE · Guía de Ejercicios: Persistencia local con Room (v1.0)

Proyecto: `EJERCICIOS-KOTLIN-ROOM` · Felipe Forte · Kotlin + Jetpack Compose + Room 2.8.5 (KSP)

Cómo leer estos apuntes: cada ejercicio tiene un recuadro **“En pocas palabras”** (qué se hace y por qué), uno de **“Aclaraciones para novatos”** (lo que suele confundir), la **explicación con el código real** del proyecto, las **pruebas hechas en el emulador** (con capturas y consultas a la base) y las **respuestas a las preguntas de la guía**. Todas las consultas a la base se hicieron copiando el archivo SQLite del emulador y consultándolo con `sqlite3`: muestran lo que realmente quedó guardado.

Glosario para empezar

Término	Qué significa, en simple
Persistir	Guardar un dato de forma que sobreviva a cerrar la app, matar el proceso o reiniciar el teléfono.
SQLite	El motor de base de datos que viene en Android. Guarda todo en **un archivo** dentro de la app (por ejemplo `notas.db`).
Tabla / fila / columna	Una tabla es como una planilla: cada *fila* es un registro (una nota) y cada *columna* un dato (título, fecha).
SQL	El idioma para hablarle a la base: `SELECT` (leer), `INSERT` (agregar), `UPDATE` (modificar), `DELETE` (borrar).
Room	Biblioteca de Android que se pone encima de SQLite: uno escribe clases de Kotlin con anotaciones y Room genera el código que habla con SQLite.
Entity	Una `data class` con `@Entity`: describe **una tabla**. Cada objeto = una fila.
DAO	Interface con `@Dao`: lista las **operaciones** sobre la tabla (insertar, buscar, borrar). Room escribe la implementación.
RoomDatabase	La clase que representa **la base entera**: qué tablas tiene, qué versión es, y da acceso a los DAO.
Clave primaria (PK)	La columna que identifica cada fila sin repetirse (acá, `id`).
Clave foránea (FK)	Una columna que apunta a la PK de otra tabla (una nota "pertenece" a una categoría).
Índice	Estructura extra que acelera buscar u ordenar por una columna, y puede impedir repetidos (índice único).
Esquema	La "forma" de la base: qué tablas y columnas tiene. Si cambia, hay que **migrar**.
Migración	Instrucciones para pasar una base instalada de una versión del esquema a la siguiente **sin perder datos**.
Transacción	Grupo de operaciones que se hacen todas o ninguna.
KSP	Herramienta que, al compilar, lee las anotaciones de Room y genera el código. Si el SQL está mal, **no compila**.
suspend / corrutina	Las operaciones de base pueden tardar: se marcan `suspend` y se llaman desde una corrutina, sin congelar la pantalla.
Flow	Un "canal" de valores. Un DAO que devuelve `Flow` vuelve a emitir la lista cada vez que la tabla cambia.
Database Inspector	Herramienta de Android Studio (App Inspection) para ver y editar la base de la app mientras corre en debug.

Organización del proyecto en carpetas

El código se organiza por capas, como en el proyecto de MVVM (y el repositorio de referencia *PokedexAcademy-MVVM-Clean*). En cada rama aparecen solo las carpetas de ese ejercicio.

```
app/src/main/java/com/example/ejercicioskotlinroom/
 ├── MainActivity.kt          # punto de entrada
 ├── NotasApp.kt              # Application: crea la ÚNICA instancia de la base (desde el Ej 4)
 ├── core/theme/              # tema de Compose
 ├── data/
 │    ├── local/              # AppDatabase, Converters, Migrations
 │    │    ├── entity/        # tablas (@Entity) y resultados de relaciones
 │    │    └── dao/           # operaciones (@Dao)
 │    └── repository/         # Repository: la API de datos que ve el ViewModel (desde el Ej 9)
 ├── domain/model/            # enums del dominio (Prioridad, EstadoPrestamo)
 └── presentation/            # una carpeta por pantalla (pantalla + ViewModel)

app/src/androidTest/java/com/example/ejercicioskotlinroom/
 └── data/local/              # tests de DAO, converters y migraciones (corren en el emulador)
app/schemas/                  # JSON del esquema de cada versión (exportSchema = true)
```

> **Diferencia con el proyecto MVVM**
>
> Siguiendo la guía de Room, la pantalla y el ViewModel trabajan directamente con las `Entity` (`NotaEntity`): no hay un modelo de dominio aparte ni casos de uso. Por eso `domain/` solo tiene los enums. Los tests de Room van en `androidTest` (necesitan Android para abrir SQLite) y se corren con `./gradlew :app:connectedDebugAndroidTest`.

Ejercicio 1 — ¿Qué se pierde cuando la app se cierra?

Consigna: Lista editable en Compose **sin Room**. Agregar ítems y probar: rotar, volver atrás, cerrar desde recientes y matar el proceso. Registrar qué sobrevive. Tabla “dato / dónde vive / dónde debería vivir”. Rama: `ej01-estado-en-memoria`

> **En pocas palabras**
> Antes de usar una base de datos hay que ver **el problema**: lo que vive en memoria se pierde. La pantalla tiene la misma lista dos veces, una con `remember` y otra con `rememberSaveable`, para comparar qué sobrevive a cada situación.

> **Aclaraciones para novatos**
> - **`remember` no guarda nada en disco:** solo evita que el valor se pierda cuando Compose redibuja. Al rotar, se pierde.
> - **`rememberSaveable`** sobrevive a la rotación (Android lo guarda en un “paquetito” de estado), pero no a que el usuario cierre la app.
> - **Ninguno de los dos es una base de datos.** Para que algo dure “para siempre” hay que escribirlo en un archivo: eso hace Room.
> - **No todo merece guardarse:** el texto a medio escribir o si un diálogo está abierto son del momento; las notas del usuario sí.

Explicación y pruebas





Antes de rotar: 2 ítems en cada lista Después de rotar: **la de remember quedó vacía**

Caso	remember	rememberSaveable
1) Rotar	❌ se pierde (probado)	✅ sobrevive (probado)
2) Volver atrás y entrar	❌	❌ la Activity se destruye del todo
3) Cerrar desde recientes	❌	❌
4) Matar el proceso desde Android Studio	❌	❌

Dato	Dónde vive ahora	Dónde debería vivir
Texto del TextField (borrador)	remember	rememberSaveable (es efímero)
Lista de elementos	remember / rememberSaveable	**Room**: es información del usuario
Preferencia visual (fondo oscuro)	remember	DataStore (preferencia chica, no una tabla)

Preguntas de la guía

1. ¿remember implica persistencia?
No. Solo conserva el valor mientras la composición vive; si la Activity se destruye, se pierde.

2. ¿Diferencia entre conservar estado ante recomposición y conservar datos al reiniciar el proceso?
Recomponer es Compose volviendo a ejecutar la función con el proceso vivo: alcanza `remember`. Reiniciar el proceso borra toda la memoria: solo sobrevive lo que está en disco (archivo, DataStore, Room).

3. ¿Todos los datos de una pantalla deberían guardarse en una base?
No. Lo que importa al usuario y debe durar (notas) va a la base; lo del momento (borrador, diálogo abierto, scroll) es estado de UI.

4. ¿Qué dato sería absurdo persistir?
El “estoy escribiendo” del TextField a medio tipear o un flag de diálogo abierto: no sirven al día siguiente y ensucian la base.

> **Criterio de finalización**
> ✅ Caso que se conserva (rememberSaveable al rotar) y caso que se pierde (remember al rotar, ambos al cerrar) documentados. Room resuelve otro problema: guardar en disco para que los datos sobrevivan a todo.

Ejercicio 2 — Primera Entity: decidir qué se guarda

Consigna: Crear `NotaEntity` revisando campo por campo, agregar una columna propia y describir la tabla resultante.
Rama: `ej02-entity`

> **En pocas palabras**
> Se define **cómo es una fila** de la tabla `notas` con una `data class`. Todavía no se guarda nada: es el "molde". Se agregó la columna `colorEtiqueta`.

> **Aclaraciones para novatos**
> - **Una instancia de `NotaEntity` = una fila.** Cada propiedad = una columna.
> - **`id = 0` no significa que todas las notas tengan id 0:** con `autoGenerate`, el 0 quiere decir "todavía no tiene"; SQLite asigna 1, 2, 3…
> - **SQLite no tiene Boolean:** Room guarda `true/false` como 1/0.
> - **Guardar datos crudos, no formateados:** la fecha va como número (`Long`) para poder ordenar; el "hace 5 minutos" se arma al mostrar.
> - **Cambiar el nombre de una propiedad cambia el nombre de la columna** y obliga a migrar una base ya instalada (Ej 14).

Explicación y pruebas

```
@Entity(tableName = "notas")
data class NotaEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val titulo: String,
    val contenido: String,
    val importante: Boolean = false,
    val creadaEn: Long,                        // milisegundos desde 1970
    val colorEtiqueta: String = "NINGUNO"     // columna agregada
)
```

Columna	Tipo SQLite	Notas
id	INTEGER PK AUTOINCREMENT	identifica la fila
titulo, contenido	TEXT NOT NULL	lo que escribe el usuario
importante	INTEGER NOT NULL	0 / 1
creadaEn	INTEGER NOT NULL	para ordenar por fecha
colorEtiqueta	TEXT NOT NULL	propia: "NINGUNO", "ROJO"…

Preguntas de la guía

1. ¿Por qué id puede empezar en 0?
Con `autoGenerate` el 0 es un centinela ("sin id todavía"); al insertar, SQLite asigna el siguiente.

2. ¿Qué parte existe para Room y qué parte es del problema?
Para Room: `@Entity`, `@PrimaryKey`, `tableName`, el id autogenerado. Del problema: título, contenido, importante, fecha, color.

3. ¿Guardarías un Painter, un Modifier o un texto formateado?
No: Painter y Modifier son objetos de la UI sin sentido fuera del proceso; un texto formateado impide ordenar y reformatear.

4. ¿Cambiar el nombre de una propiedad puede afectar una base instalada?
Sí: cambia el esquema. Hay que subir la versión y migrar, o fijar el nombre con `@ColumnInfo(name = ...)`.

> **Criterio de finalización**
> ✅ Compila con una `@Entity` válida y PK clara; la explicación de qué representa una fila está en el código.

Ejercicio 3 — DAO: escribir y preguntar a la base

Consigna: Crear `NotaDao`: insertar, listar todas, borrar todas, más una consulta por id y otra de importantes. Desafío: contar sin cargar filas.
Rama: `ej03-dao`

> **En pocas palabras**
> El DAO es la **lista de cosas que se le pueden pedir a la tabla**. Uno escribe la firma y el SQL; Room genera el código. Todavía no se conecta a la pantalla.

> **Aclaraciones para novatos**
> - **El DAO es una `interface`**: no se implementa a mano. KSP genera la clase al compilar.
> - **`@Query` lleva SQL dentro de un String**, y Room lo verifica: si una columna no existe, el proyecto no compila.
> - **`insertar` devuelve un `Long`:** el id que le tocó a la fila nueva.
> - **Buscar por id puede no encontrar nada:** por eso devuelve `NotaEntity?` (puede ser null).
> - **`suspend`** porque leer o escribir en disco puede tardar; Room no deja hacerlo en el hilo principal.

Explicación y pruebas

```
@Dao
interface NotaDao {
    @Insert suspend fun insertar(nota: NotaEntity): Long
    @Query("SELECT * FROM notas ORDER BY creadaEn DESC")
    suspend fun obtenerTodasUnaVez(): List<NotaEntity>
    @Query("DELETE FROM notas") suspend fun borrarTodas()
    @Query("SELECT * FROM notas WHERE id = :id LIMIT 1")
    suspend fun obtenerPorId(id: Int): NotaEntity?            // propia
    @Query("SELECT * FROM notas WHERE importante = 1 ORDER BY creadaEn DESC")
    suspend fun obtenerImportantes(): List<NotaEntity>        // propia
    @Query("SELECT COUNT(*) FROM notas")
    suspend fun contarNotas(): Int                            // desafío
}
```

Preguntas de la guía

1. ¿Qué interpreta Room y qué SQLite?
Room: las anotaciones y la firma (genera código, convierte Entity ⇄ fila, valida el SQL al compilar). SQLite: el texto SQL, que ejecuta contra el archivo.

2. ¿Por qué insertar() devuelve Long?
Devuelve el rowId de la fila insertada, que con PK autogenerada es el id nuevo.

3. ¿Y si la consulta por id no encuentra nada?
No hay fila: el retorno razonable es nullable (`NotaEntity?`) y null significa “no existe”.

4. ¿Por qué no bloquear el hilo principal?
Disco puede tardar: congelaría la UI (ANR). Son `suspend` y Room usa su propio hilo de IO.

Criterio de finalización

✅ 6 operaciones útiles, todas sobre columnas que existen en `NotaEntity` (si no, no compilaría).

Ejercicio 4 — Crear la base y mirarla por dentro

Consigna: Crear `AppDatabase`, construir **una única instancia** para la app, insertar al menos tres notas y verlas en Database Inspector.
Rama: `ej04-database`

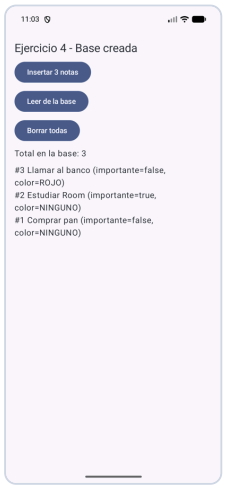
> **En pocas palabras**
> Se crea la base real (`notas.db`) y se la deja en la clase `Application`, que vive mientras vive la app. Una pantalla de prueba inserta 3 notas y las lee.

> **Aclaraciones para novatos**
> - **`AppDatabase` define; `Room.databaseBuilder(...).build()` abre.** La primera es la descripción; la segunda crea/abre el archivo.
> - **Una sola instancia:** crear la base en cada recomposición abriría conexiones sin parar y los Flow de una no verían lo que escribe otra.
> - **`by lazy`** crea la base recién la primera vez que se usa, y una sola vez.
> - **`version = 1`** es la versión del esquema: se guarda dentro del archivo y sirve para saber cuándo migrar.
> - **Database Inspector no viaja en la app:** es una herramienta de Android Studio para debug.

Explicación y pruebas

```
@Database(entities = [NotaEntity::class], version = 1, exportSchema = true)
abstract class AppDatabase : RoomDatabase() {
    abstract fun notaDao(): NotaDao
}

class NotasApp : Application() {                // registrada en el Manifest
    val database: AppDatabase by lazy {
        Room.databaseBuilder(applicationContext, AppDatabase::class.java, "notas.db").build()
    }
}
```



Pantalla de prueba: “Insertar 3 notas” y “Leer de la base”

Consulta al archivo real copiado del emulador:

```
sqlite> SELECT name FROM sqlite_master WHERE type='table';
android_metadata | notas | sqlite_sequence | room_master_table
sqlite> PRAGMA user_version;          -> 1
sqlite> SELECT id, titulo, importante, colorEtiqueta FROM notas;
 1  Comprar pan       0  NINGUNO
 2  Estudiar Room     1  NINGUNO
 3  Llamar al banco   0  ROJO
```

Preguntas de la guía

1. ¿Qué representa version = 1?
La versión del esquema, guardada en el archivo (`PRAGMA user_version`). Al cambiar el esquema se sube y se migra.

2. ¿Relación entre AppDatabase, el archivo SQLite y el DAO?
Archivo = los datos; AppDatabase = la puerta de entrada que conoce el esquema; DAO = las operaciones que se obtienen de ella.

3. ¿Por qué una única instancia?
Es cara, cada una abre su conexión, y la invalidación de los Flow es por instancia.

4. ¿Database Inspector es parte de la app final?
No: es de Android Studio, no se empaqueta en el APK.

> **Criterio de finalización**
> ✅ La tabla tiene 3 filas reales (comprobado arriba). Esquema: `data/local/AppDatabase.kt` ; apertura física: `NotasApp.kt` .

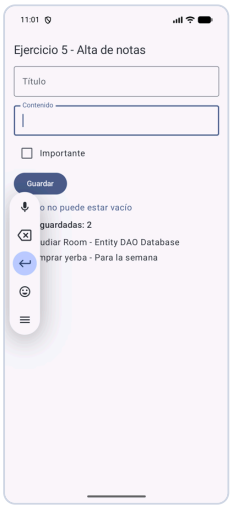
Ejercicio 5 — Alta de datos desde Compose

Consigna: Formulario con título, contenido, importante, Guardar y mensaje. Insertar fuera del hilo principal. Validar: título no vacío, recortar espacios, evitar duplicados por doble toque.
Rama: `ej05-alta-compose`

> **En pocas palabras**
> Primera pantalla conectada a Room: el usuario escribe una nota y se inserta en la tabla. Se valida antes de guardar y se muestra el resultado.

> **Aclaraciones para novatos**
> - **`trim()`** saca los espacios de los extremos: “ Hola ” queda “Hola”.
> - **`creadaEn` lo pone el código** (`System.currentTimeMillis()`) justo antes de insertar: no es un campo del formulario.
> - **Doble toque:** mientras se guarda, el botón se deshabilita (`guardando = true`). Sin eso, dos toques rápidos crean dos filas iguales.
> - **`rememberCoroutineScope()`** da corrutinas atadas a la pantalla; se cancelan si la pantalla desaparece. Más adelante el lugar correcto es el ViewModel.
> - **“Silenciosamente” es lo prohibido:** si el título está vacío no se guarda *y se avisa*.

Explicación y pruebas



Título vacío → “El título no puede estar vacío”;
las 2 notas válidas quedaron guardadas

```
sqlite> SELECT id, titulo, contenido FROM notas;
 1  Comprar yerba   Para la semana
 2  Estudiar Room   Entity DAO Database      (el intento con título vacío no está)
```

Preguntas de la guía

1. ¿Qué dato de la UI no debería terminar en la tabla?
El estado de la pantalla: texto a medio escribir, el mensaje “Guardada”, el flag `guardando`.

2. ¿Dónde se genera creadaEn y por qué?
Al guardar, con `System.currentTimeMillis()`: tiene que ser el momento real del guardado.

3. ¿Qué señal indica que la inserción terminó bien?
El `Long` que devuelve `insertar()`: un id > 0. Se muestra en el mensaje.

4. ¿Riesgo de lanzar corrutinas en cualquier lado?
Que sigan vivas con la pantalla muerta o se lancen varias veces (duplicados). Hay que usar un scope atado al ciclo de vida.

> **Criterio de finalización**
> ✅ Se cargan notas desde la UI y siguen al reiniciar; un título inválido no se inserta y se avisa.

Ejercicio 6 — Leer con Flow y dejar que la pantalla reaccione

Consigna: El DAO expone `Flow<List<NotaEntity>>`; la pantalla lo muestra en una `LazyColumn` que se actualiza sola. Desafío: estado vacío sin lista paralela.
Rama: `ej06-flow`

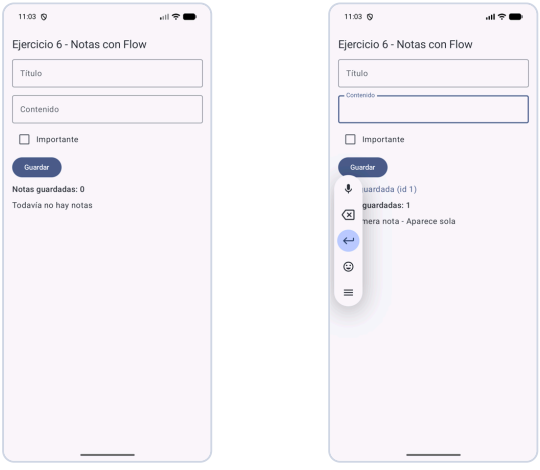
> **En pocas palabras**
> En vez de pedir la lista una vez, la pantalla se **suscribe**: cada vez que la tabla cambia, Room vuelve a ejecutar la consulta y la lista se redibuja sola. Ya no hace falta un botón “Recargar”.

> **Aclaraciones para novatos**
> - **`List` es una foto; `Flow` es una película:** la foto no se entera de cambios, la película sí.
> - **Una función que devuelve `Flow` no lleva `suspend`:** no espera un resultado, devuelve un canal al que uno se suscribe.
> - **`collectAsStateWithLifecycle()`** convierte el Flow en algo que Compose observa, y deja de escuchar en segundo plano.
> - **Estado vacío sin inventar nada:** “Todavía no hay notas” sale de `notas.isEmpty()`, la misma lista.

Explicación y pruebas

```
@Query("SELECT * FROM notas ORDER BY creadaEn DESC")
fun observarNotas(): Flow<List<NotaEntity>>

// en la pantalla
val notas by dao.observarNotas().collectAsStateWithLifecycle(initialValue = emptyList())
if (notas.isEmpty()) Text("Todavía no hay notas")
LazyColumn { items(notas, key = { it.id }) { ... } }
```



Base vacía: estado vacío Después de Guardar: aparece sola

Cadena completa: la tabla cambia → Room invalida el Flow → emite la lista nueva → `collectAsStateWithLifecycle` actualiza el State → la `LazyColumn` se recompone.

Preguntas de la guía

1. ¿Diferencia entre Flow<List> y List?
List es el resultado de un momento; Flow emite una lista nueva cada vez que la tabla cambia.

2. ¿Room vuelve a ejecutar la consulta?
Sí, mientras alguien recolecte el Flow, cada vez que se invalida una tabla que la consulta lee.

3. ¿Es lo mismo un Flow que un estado de Compose?
No: Flow es de corrutinas; el State de Compose avisa que hay que recomponer. collectAsState… los conecta.

4. ¿Qué hace collectAsStateWithLifecycle?
Recolecta, guarda el último valor en un State, dispara recomposición y pausa en segundo plano.

> **Criterio de finalización**
> ✅ La lista reacciona a inserciones sin recarga manual (probado).

Ejercicio 7 — CRUD completo: editar y eliminar

Consigna: Agregar `@Update` y `@Delete`; editar una nota existente y eliminar con confirmación. Comprobar que editar no crea otra fila. Desafío: Deshacer.
Rama: `ej07-crud`

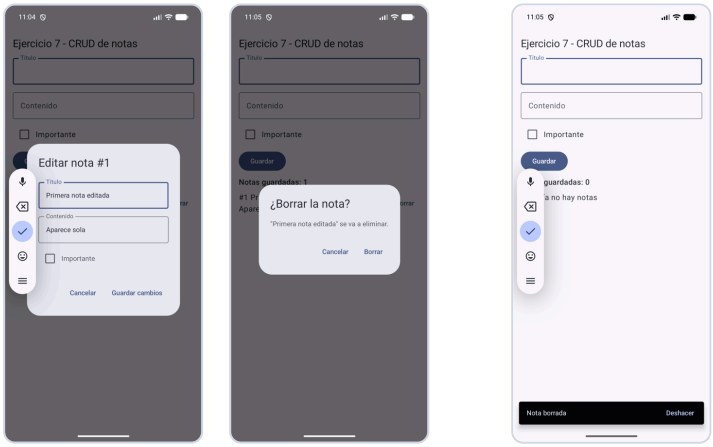
> **En pocas palabras**
> Se completa el **CRUD** (Crear, Leer, Actualizar, Borrar). Editar modifica la **misma fila** (mismo id); borrar pide confirmación y ofrece “Deshacer”.

> **Aclaraciones para novatos**
> - `@Update` **busca la fila por la clave primaria** del objeto: por eso se edita con `nota.copy(titulo = ...)`, que conserva el id.
> - **Si se arma una Entity nueva con id 0 y se manda a** `@Update`**, no actualiza nada** (no existe la fila 0).
> - **Deshacer** guarda en memoria la nota borrada (con su id) y la vuelve a insertar: reaparece igual.
> - **Ese “Deshacer” no es persistente:** si se cierra la app antes de tocarlo, la nota se pierde.

Explicación y pruebas

```
@Update suspend fun actualizar(nota: NotaEntity): Int   // filas modificadas
@Delete suspend fun eliminar(nota: NotaEntity): Int

dao.actualizar(nota.copy(titulo = nuevoTitulo))       // mismo id → misma fila
```



Editar: diálogo Borrar: confirmación Borrada: Snackbar “Deshacer”

```
Editar "Primera nota" → id sigue siendo 1, sigue habiendo 1 fila
Borrar → SELECT count(*) = 0 ;  Deshacer → vuelve "1 | Primera nota editada"
```

Preguntas de la guía

1. ¿Cómo sabe @Update qué fila modificar?

Por la PK del objeto: `UPDATE notas SET … WHERE id = :id`.

2. ¿Y si el objeto editado tiene otro id?
Pisaría otra fila; con un id inexistente no modifica nada (devuelve 0).

3. ¿Borrar directo o pedir confirmación?
Confirmar (o deshacer): borrar es destructivo y un toque accidental es fácil. Se hicieron las dos.

4. ¿Qué podría vivir solo en memoria?
Qué nota se edita, si el diálogo está abierto, el borrador y la nota guardada para Deshacer.

> **Criterio de finalización**
> ✅ Crear, leer, editar y eliminar sobre la misma base; la edición conserva el id (probado con sqlite).

Ejercicio 8 — Consultas que empiezan a parecerse a una app real

Consigna: Buscador por título, solo importantes, orden por fecha y contadores (total e importantes), todo resuelto en SQL.
Rama: `ej08-consultas`

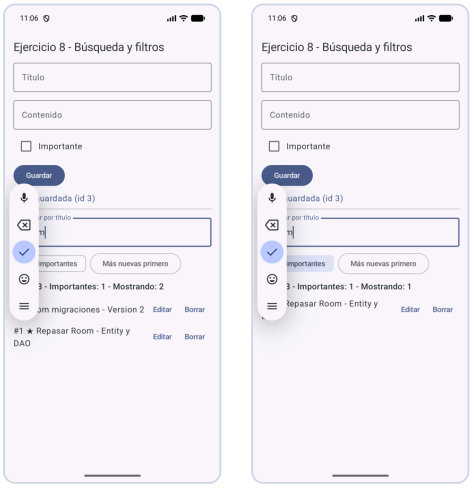
> **En pocas palabras**
> La búsqueda, los filtros, el orden y los contadores los hace **SQLite**, no Kotlin. A la app solo llega el resultado.

> **Aclaraciones para novatos**
> - `LIKE '%' || :texto || '%'` significa “contiene el texto”. `||` une textos en SQL.
> - `:texto` es un parámetro: Room lo pasa por separado (no lo pega en el SQL), lo que evita inyección SQL.
> - `COUNT(*)` cuenta filas en la base: los contadores no se llevan “a mano”.
> - **Filtrar en SQL** evita cargar toda la tabla en memoria para tirar la mayoría de las filas.
> - **Tests del DAO:** desde esta rama hay tests en `androidTest` que usan una base *en memoria* descartable.

Explicación y pruebas

```
@Query("""
    SELECT * FROM notas
    WHERE titulo LIKE '%' || :texto || '%'
      AND (:soloImportantes = 0 OR importante = 1)
    ORDER BY CASE WHEN :ascendente = 1 THEN creadaEn END ASC,
             CASE WHEN :ascendente = 0 THEN creadaEn END DESC
""")
fun filtrar(texto: String, soloImportantes: Boolean, ascendente: Boolean):
Flow<List<NotaEntity>>

@Query("SELECT COUNT(*) FROM notas") fun observarCantidad(): Flow<Int>
```



Buscar “Room”: 2 de 3 + Solo importantes: 1

Tests instrumentados (`NotaDaoTest`): **4/4** ✅ en el emulador.

Preguntas de la guía

1. ¿Ventaja de filtrar en SQL?
El motor descarta filas (y puede usar índices); con `filter` se cargan todas a memoria.

2. ¿Cómo entra :texto?
Como argumento enlazado (bind): Room no concatena el String, evita inyección.

3. ¿Ordenar en SQL vs en memoria?
En SQL ordena el motor (puede usar un índice) y el Flow ya llega ordenado.

4. ¿Qué consulta se beneficiaría de un índice?
El `ORDER BY creadaEn`. `LIKE '%x%'` con comodín adelante no usa índices comunes.

> **Criterio de finalización**
> ✅ Búsqueda y filtro respaldados por consultas del DAO; contadores con `COUNT(*)`.

Ejercicio 9 — Poner orden: Repository, ViewModel y Room

Consigna: Reorganizar: Compose UI → ViewModel → Repository → DAO → Room. El Repository recibe el DAO por constructor; la UI no llama al DAO. Desafío: interfaz + implementación.
Rama: `ej09-repository-viewmodel`

> **En pocas palabras**
> La pantalla deja de hablar con la base. Ahora pide cosas al **ViewModel** (“guardar nota”), que valida y se lo pasa al **Repository**, que es quien usa el DAO. Mismo comportamiento, mejor ordenado.

> **Aclaraciones para novatos**
> - **El ViewModel guarda el estado y sobrevive a la rotación**; la pantalla solo dibuja y avisa eventos.
> - **El Repository habla “en idioma de notas”** (agregar, observar) y esconde que atrás hay SQL.
> - **La validación (“título vacío”) la decide el ViewModel**, no la pantalla ni el DAO.
> - `flatMapLatest` : cuando cambia el texto de búsqueda, deja la consulta anterior y empieza la nueva.
> - `stateIn` convierte el Flow en StateFlow con un valor inicial para que la pantalla siempre tenga algo que mostrar.

Explicación y pruebas

```
IDA:    Guardar → viewModel.guardarNota() → valida → repositorio.agregar() → dao.insertar() → SQLite
VUELTA: SQLite cambia → Room re-emite el Flow → repositorio.observarNotas() →
        flatMapLatest + stateIn en el ViewModel → collectAsStateWithLifecycle → LazyColumn

interface NotaRepository { fun observarNotas(...): Flow<List<NotaEntity>>; suspend fun agregar(...) ... }
class NotaRepositoryImpl(private val notaDao: NotaDao) : NotaRepository { ... }   // recibe el DAO
```

La interfaz está en `data/repository/NotaRepository.kt` y la implementación en `NotaRepositoryImpl.kt` . Tests: 4/4 ✅ .

Preguntas de la guía

1. ¿Qué gana y qué cuesta la separación?
Gana: la UI no conoce el DAO, cada capa una responsabilidad, estado que sobrevive a la rotación, más fácil de probar. Cuesta: más archivos y métodos que solo delegan.

2. ¿El Repository tiene que existir siempre?
No; si solo reenvía llamadas no aporta. Acá genera `creadaEn` , arma la entidad y sería el lugar para una fuente remota.

3. ¿Quién decide si un título vacío es válido?
El ViewModel: es una regla de entrada de usuario y arma el mensaje.

4. Con red y Room, ¿quién coordina?
El Repository: Room como fuente de verdad y la red para refrescar.

Criterio de finalización

✅ La pantalla funciona igual que antes sin tocar el DAO.

Ejercicio 10 — Restricciones, índices y conflictos

Consigna: Agregar `codigoExterno` que no se repita (índice único), elegir estrategia de conflicto y un índice no único justificado.
Rama: `ej10-restricciones-indices`

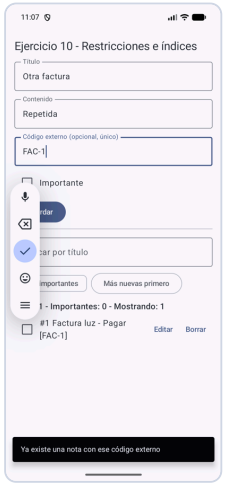
> **En pocas palabras**
> Se le pide a la **base** que proteja una regla: no puede haber dos notas con el mismo código externo. Si se intenta, SQLite lo rechaza y la app avisa.

> **Aclaraciones para novatos**
> - **Índice único ≠ clave primaria:** la PK identifica la fila; el único solo impide repetidos y admite NULL.
> - **Varias notas sin código (NULL) no chocan**, pero `""` sí es un valor: por eso el texto en blanco se convierte en null.
> - **IGNORE** no inserta y devuelve -1; **ABORT** lanza excepción; **REPLACE** borra la fila vieja y mete la nueva (peligroso).
> - **Los índices no son gratis:** aceleran lecturas pero ocupan espacio y hacen más lentas las escrituras.
> - **Ojo en esta rama:** el esquema cambió sin subir la versión (todavía no hay usuarios): hay que borrar los datos de la app antes de probar. Desde el Ej 14, se migra.

Explicación y pruebas

```
@Entity(tableName = "notas",
        indices = [Index(value = ["codigoExterno"], unique = true),
                   Index(value = ["creadaEn"])])          // todas las consultas ordenan por
creadaEn

@Insert(onConflict = OnConflictStrategy.IGNORE)
suspend fun insertar(nota: NotaEntity): Long            // -1 si viola la restricción
```



Segundo “FAC-1”: “Ya existe una nota con ese código externo”

```
sqlite> SELECT id, titulo, codigoExterno FROM notas;     ->  1 | Factura luz | FAC-1   (una
sola)
sqlite> índices de notas:
CREATE UNIQUE INDEX index_notas_codigoExterno ON notas (codigoExterno)
CREATE INDEX        index_notas_creadaEn      ON notas (creadaEn)
```

Tests: **8/8** ✅ (incluye insertar dos veces el mismo código con ABORT, IGNORE y REPLACE).

Preguntas de la guía

1. ¿Índice único = clave primaria?
No: la PK identifica la fila, es una sola y no admite NULL; un índice único solo evita repetidos.

2. ¿Problema de REPLACE por comodidad?
Pierde la fila anterior sin avisar, cambia el id y puede disparar borrados en cascada.

3. ¿Los índices mejoran todo gratis?
No: cuestan espacio y escrituras más lentas.

4. ¿Cómo comprobar que la restricción se cumple?
Con un test que inserta dos veces el mismo valor (y en la base real, arriba).

> **Criterio de finalización**
> ✅ Restricción de unicidad demostrada con prueba; índice sobre `creadaEn` justificado porque todas las consultas ordenan por esa columna.

Ejercicio 11 — Dos tablas: relación uno a muchos

Consigna: Agregar `CategoriaEntity`; una categoría tiene muchas notas. Clave foránea, clase con `@Embedded` + `@Relation`. Desafío: política de borrado.
Rama: `ej11-relacion-uno-a-muchos`

> **En pocas palabras**
> Aparece una segunda tabla, `categorias`. Cada nota puede apuntar a una categoría. Si se borra una categoría, sus notas **no se borran**: quedan “sin categoría”.

> **Aclaraciones para novatos**
> - **La clave foránea va del lado “muchos”:** `notas.categoriaId` apunta a `categorias.id`.
> - **No se guarda una lista de notas dentro de la categoría:** una columna guarda un solo valor; para eso está la otra tabla.
> - `SET_NULL`: al borrar la categoría, las notas quedan con `categoriaId = NULL`. `CASCADE` las borraría; `RESTRICT` impediría borrar.
> - `CategoriaConNotas` **no es una tabla:** es una clase de lectura que Room arma juntando las dos.
> - `categoriaId` **es nullable:** una nota puede existir sin categoría.

Explicación y pruebas

```
ForeignKey(entity = CategoriaEntity::class, parentColumns = ["id"],
           childColumns = ["categoriaId"], onDelete = ForeignKey.SET_NULL)

data class CategoriaConNotas(
    @Embedded val categoria: CategoriaEntity,
    @Relation(parentColumn = "id", entityColumn = "categoriaId") val notas: List<NotaEntity>
)
```



Panel de categorías con sus notas

```
Antes:  1 Parcial DA1 → Facultad | 2 TP Rabbit → Facultad | 3 Lavar ropa → Casa
Borrar "Facultad":
sqlite> SELECT id, titulo, categoriaId FROM notas;
 1  Parcial DA1   (NULL)     ← la nota sigue, sin categoría
 2  TP Rabbit     (NULL)
 3  Lavar ropa    2
```

Tests: **12/12** ✅ (incluye el efecto de SET_NULL).

Preguntas de la guía

1. ¿Por qué no guardar la lista de notas en una columna?
Rompe el modelo relacional: no se puede consultar ni filtrar, no hay integridad y hay que reescribir toda la lista.

2. ¿Dónde queda la clave foránea?
En `notas.categoriaId`, el lado "muchos".

3. ¿Qué pasa con las notas al borrar una categoría?
Quedan sin categoría (SET_NULL): decisión de producto para no perder notas.

4. ¿Entidad vs clase de resultado relacional?
La entidad tiene tabla y se inserta; `CategoriaConNotas` solo se lee.

> **Criterio de finalización**
> ✅ Dos tablas relacionadas, consulta de categoría con sus notas y efecto del borrado decidido y probado.

Ejercicio 12 — TypeConverters: cuando SQLite no conoce tu tipo

Consigna: Reemplazar un campo primitivo por un tipo más expresivo (enum `Prioridad`), crear el `@TypeConverter` y verificar qué se guardó.
Rama: `ej12-typeconverters`

> **En pocas palabras**
> SQLite no sabe qué es un enum. El **TypeConverter** le explica a Room cómo pasar `Prioridad.ALTA` a texto (`"ALTA"`) al guardar, y al revés al leer.

> **Aclaraciones para novatos**
> - **SQLite solo tiene 4 tipos:** INTEGER, REAL, TEXT, BLOB. Todo lo demás hay que convertirlo.
> - **Se guarda el nombre, no el número de orden:** si se reordenara el enum, los ordinales cambiarían de significado.
> - **Renombrar una constante rompe datos viejos:** las filas siguen diciendo “ALTA”. Haría falta una migración con UPDATE.
> - **Hay que registrar el converter** con `@TypeConverters(Converters::class)` en la base.

Explicación y pruebas

```
enum class Prioridad { BAJA, MEDIA, ALTA }      // domain/model/Prioridad.kt

class Converters {
    @TypeConverter fun prioridadAString(valor: Prioridad): String = valor.name
    @TypeConverter fun stringAPrioridad(valor: String): Prioridad =
        Prioridad.entries.firstOrNull { it.name == valor } ?: Prioridad.MEDIA
}
```



Selector de prioridad en el formulario

```
sqlite> SELECT id, titulo, prioridad, typeof(prioridad) FROM notas;
 1  Parcial DA1    ALTA   text
 2  TP Rabbit      MEDIA  text
 3  Leer libro     BAJA   text
```

Tests: **14/14** ✅ (`ConvertersTest` verifica el valor físico).

Preguntas de la guía

1. ¿Room guarda el enum completo?
No: solo un String con el nombre de la constante.

2. ¿Y si ALTA se renombra a URGENTE?
Las filas viejas dicen “ALTA”: hace falta migrar (`UPDATE … SET prioridad = 'URGENTE'`).

3. ¿Fecha como número o como texto?
Número: compacto, ordena bien, sin ambigüedad de zona. Texto: legible pero ocupa más y depende del formato.

4. ¿Qué resuelve el TypeConverter y qué no?
Resuelve la traducción en ambos sentidos; no resuelve la evolución de los datos ni valida el valor guardado.

> **Criterio de finalización**
> ✅ Un tipo no soportado por SQLite se guarda y recupera; el valor físico (`text`) se verificó en la base.

Ejercicio 13 — Transacciones: dos pasos que deberían ser uno

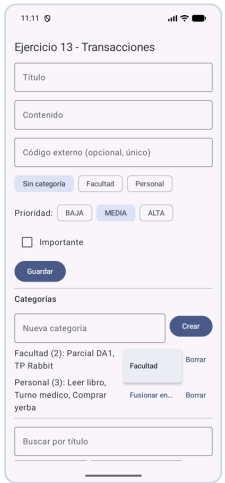
Consigna: Operación de varios pasos: **fusionar** categorías (mover notas y borrar la vacía). Versión sin transacción vs con `@Transaction`; lectura relacional transaccional.
Rama: `ej13-transacciones`

> **En pocas palabras**
> Fusionar son dos escrituras. Si la app se cae entre una y otra, queda un estado a medias. Con una **transacción**, o se hacen las dos o ninguna.

> **Aclaraciones para novatos**
> - **Atómico = todo o nada.** Si falla un paso, se deshace todo (rollback).
> - **SQLite puede quedar "válido" pero la app no:** una categoría vacía que debía borrarse no rompe ninguna regla de SQLite, pero sí la del negocio.
> - **Las transacciones también sirven para leer:** categoría + sus notas son dos SELECT que deben ver la misma foto.
> - **Para probarlo se fuerza un fallo** entre los dos pasos (`fallarEntrePasos = true`, solo en tests).
> - **El DAO pasó a clase abstracta** para poder tener un método con cuerpo y `@Transaction`.

Explicación y pruebas

```
@Transaction
open suspend fun fusionar(origen: CategoriaEntity, destinoId: Int, fallarEntrePasos: Boolean =
false) {
    moverNotas(origen.id, destinoId)                  // paso 1: UPDATE notas
    if (fallarEntrePasos) throw IllegalStateException("Fallo forzado entre los dos pasos")
    eliminar(origen)                                  // paso 2: DELETE categoría
}
```



"Fusionar en…" Facultad

```
Fusionar "Personal" (3 notas) en "Facultad" (2 notas):
sqlite> SELECT * FROM categorias;              -> 1 | Facultad   (Personal ya no existe)
sqlite> SELECT id, titulo, categoriaId FROM notas;   -> las 5 notas con categoriaId = 1
```

Tests: **17/17** ✅ . Con fallo forzado: *con* transacción las notas vuelven a su categoría (rollback); *sin* transacción quedan movidas y la categoría origen sigue existiendo.

Preguntas de la guía

1. ¿Qué significa atómica?
Que se hace completa o no se hace.

2. ¿Una transacción sirve solo para escrituras?
No: también para lecturas que necesitan consistencia entre varias consultas.

3. ¿Qué error deja la base válida para SQLite pero inválida para la app?
Notas movidas y la categoría origen sin borrar: ninguna restricción se rompe, pero “fusionar” quedó a medias.

4. ¿Qué conviene probar forzando un fallo?
El punto exacto entre los dos pasos: con transacción, rollback; sin ella, estado a medias.

> **Criterio de finalización**
> ✅ Operación de dos pasos transaccional y explicación del estado inconsistente evitado (probado con fallo forzado).